UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ACTIVIDAD 2.1

# Modelado de problemas de búsqueda

Dos problemas representados mediante grafos ponderados

| | |
|---|---|
| **Universidad** | Politécnica de Atlautla |
| **Programa** | Ing. en Tecnologías de la Información e Innovación Digital |
| **Asignatura** | Fundamentos de Inteligencia Artificial |
| **Grupos** | 171 y 172 · Cuatrimestre 7 |
| **Docente** | Mtro. Javier García Lara |
| **Periodo** | 12 al 26 de octubre de 2026 |

**Alumno:** Escribe aquí tu nombre completo.  **Sección:** 171 / 172.

**Nota de entrega:** Revisa los datos personales, ejecuta el código y sustituye cualquier resultado de ejemplo por el resultado de tu propia ejecución antes de entregar.

# 1. Introducción y objetivo

Un problema de búsqueda se define mediante un estado inicial, acciones posibles, un modelo de transición, una prueba de meta y un costo. Representar un problema como grafo permite comparar rutas y justificar la solución.

Objetivo: modelar dos situaciones reales simplificadas como grafos ponderados y resolverlas con búsqueda de costo uniforme, que expande primero el estado con menor costo acumulado.

# 2. Problema A: recorrido dentro de un campus

Situación: una persona entra al campus y quiere llegar a la salida pasando por pasillos cuyo costo representa distancia o tiempo estimado.

Estado inicial: Entrada. Meta: Salida. Acciones: desplazarse a un lugar conectado. Transición: seguir una arista del grafo. Costo: unidades de distancia/tiempo asignadas a cada conexión. Supuesto: costos no negativos y mapa estático.

```
campus = {
 'Entrada': [('Biblioteca', 2), ('Laboratorio', 4)],
 'Biblioteca': [('Cafeteria', 3), ('Aulas', 4)],
 'Laboratorio': [('Aulas', 2), ('Taller', 3)],
 'Cafeteria': [('Salida', 5)],
 'Aulas': [('Salida', 2)],
 'Taller': [('Salida', 4)], 'Salida': []
}
```

Resultado esperado con búsqueda de costo uniforme: Entrada → Biblioteca → Aulas → Salida, costo total 8. La alternativa Entrada → Laboratorio → Aulas → Salida también cuesta 8; por tanto, hay más de una ruta óptima.

# 3. Problema B: entrega de un paquete

Situación: un repartidor sale del almacén y debe llegar al destino con el menor costo acumulado. Los nodos representan puntos de referencia; los pesos representan minutos ficticios de recorrido.

Estado inicial: Almacen. Meta: Destino. Acciones: tomar una calle conectada. Costo: minutos estimados. Restricciones: solo se permiten aristas explícitas y se asume que no hay cierres de calles.

```
entregas = {
 'Almacen': [('Calle_A', 2), ('Calle_B', 5)],
 'Calle_A': [('Cruce', 2), ('Parque', 4)],
 'Calle_B': [('Cruce', 1), ('Hospital', 3)],
 'Cruce': [('Hospital', 2), ('Destino', 5)],
 'Parque': [('Destino', 3)],
 'Hospital': [('Destino', 2)], 'Destino': []
}
```

Ruta óptima: Almacen → Calle_A → Cruce → Hospital → Destino, costo 8 minutos ficticios. También puede existir otra ruta de costo equivalente dependiendo de las aristas y pesos. El programa imprime la ruta realmente calculada.

# 4. Algoritmo y complejidad

```
def costo_uniforme(grafo, inicio, meta):
    frontera = [(0, inicio, [inicio])]
    mejor_costo = {inicio: 0}
    while frontera:
        costo, estado, camino = heapq.heappop(frontera)
        if estado == meta:
            return costo, camino
        if costo > mejor_costo.get(estado, float('inf')):
            continue
        for vecino, peso in grafo.get(estado, []):
            nuevo = costo + peso
            if nuevo < mejor_costo.get(vecino, float('inf')):
                mejor_costo[vecino] = nuevo
                heapq.heappush(frontera, (nuevo, vecino, camino + [vecino]))
    return None, []
```

Con una cola de prioridad, la búsqueda de costo uniforme puede expresarse aproximadamente como O((V + E) log V) para una implementación eficiente, donde V es el número de estados y E el número de conexiones. Almacenar la frontera y los padres requiere memoria O(V) en la formulación habitual.

# 5. Resultados, validación y límites

El archivo adjunto imprime la ruta y el costo de cada caso. Se recomienda probar: (a) inicio igual a meta; (b) meta inalcanzable; (c) arista con costo cero; (d) dos rutas con igual costo; (e) grafo sin conexiones.

La búsqueda de costo uniforme es apropiada si los costos no son negativos. Si todos los pasos cuestan lo mismo, BFS puede ser más sencillo. El mapa didáctico no representa un campus o ciudad real y los pesos deben reemplazarse por mediciones antes de usarse en campo.

## 6. Conclusiones

El modelado explícito evita confundir ubicaciones, acciones y costos. La solución no depende de adivinar una ruta, sino de explorar sistemáticamente las alternativas y conservar el menor costo conocido. Para problemas reales se debe actualizar el grafo y validar los pesos.

## 7. Referencias APA 7

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Bo■hm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science Publishers.

Python Software Foundation. (s. f.). *heapq — Heap queue algorithm*. https://docs.python.org/3/library/heapq.html

**Declaración de uso de IA:** Se utilizó una herramienta de IA generativa como apoyo para organizar ideas, proponer ejemplos y revisar la redacción. El alumno debe verificar las fuentes, comprender el código, ejecutar las pruebas y adaptar el contenido a su trabajo.